Flores Diaz Diego Adair 323294506.

Pregunta 0.1 ¿Qué es la abstracción al modelar un objeto? Da un dato que conservarías y otro que omitirías según el contexto.

- La abstracción es conservar unicamente las caracteristicas esenciales del objeto que son importantes para el problema. Por ejemplo al modelar un carro, tomamos el nivel de gasolina, pero ignoramos el color.

Pregunta 0.2 ¿Para qué sirve el constructor y en qué se distingue de un método común en su declaración?

- Sirve para crear un nuevo objeto, este no lleva ningún tipo de retorno, ni void, y tiene que llevar el mismo nombre que la clase.

Pregunta 0.3 ¿Qué impide final en una variable? Si guarda una referencia a un arreglo. ¿impide cambiar sus elementos?

- Impide que se le asigne una nueva referencia (dirección de memoria) a la que apunta. Pero el contenido sí se puede modificar.

Pregunta 0.4 ¿Por qué == no basta para comparar el contenido de dos strings? Explica que usarías para comparar textos?

- El == no compara Strings, solo dirección de memoria, el que se usa es equals este si compara espacios, mayúsculas y minúsculas.

1. ¿Qué imprime?

| i | Fila[i]/2 | tabla[0] | total += tabla[0][i] | total | System.Out. |
|---|---|---|---|---|---|
| 0 | 9/2 | 4 | 0+4 | 4, | 0: 4 |
| 1 | 4/2 => | 2 | 4+2 | 6 | 1: 6 |
| 2 | 7/2 | 3 | 6+3 | 9 | 2: 9 |

El System (double) (total/2) Da eso por que divide el 9/2 pero total es int, entonces esto toma la parte entera en este caso 4, y sale 4.0 por el cast a double y asi tambien en el Fila[i]/2

2 a) ¿Compila, truena, funciona?

```
1 int[] datos = {3, 6};
2 int total = 0
3 for (int i = 0; i <= datos.length; i++)
4     total += datos[i];
5 System.out.println(total);
```

La linea que causa el error es la 5. en el for se usa el i <= datos.length. Los arreglos inician en indice cero. Cuando i llega al valor datos.length que es 2 el programa quiere acceder a datos[2]. Pero este no existe por eso el código truena pero si compila. Es la regla de limite de arreglos.

b) ¿Compila, truena, funciona?

```
1 Integer piezas = 6;
2 System.out.println(piezas + 2);
3 System.out.println("P" + piezas);
```

Funciona y sala 8 y P6; Porque se declara como tipo envoltorio con valor 6. En la linea 2 se usa el operador "+" por lo que el 6 se vuelve int(6) y se realiza la suma normal. En la linea 3 de igual manera pero ahora son cadenas, por eso Java desempaqueta el 6 y ahora lo representa como texto y se concatena la "P" + el "6" dando "P6".

c) ¿Compila, truena o funciona?

```
1 Slatic int medir (int valor) {
2 if (valor > 0) return valor;
3 }
```

El método esta declarado para siempre devolver un int, pero solo tiene un retorno, cuando valor es mayor a 0. Si valor es cero o negativo, el programa llega al final y no encuentra ningún return. y si se declara como que regresará un int. todo tiene que return un valor int.

## B3. Modelado, base, reto y justificación.

```
Public class Lampara {
    enum Estado { APAGADO, ENCENDIDO, PARPADEO }
    String nombreLampara;
    Estado estado;
    Static int numLamps = 0;
    Static final String ETIQUETA = "LAMPARA";

Lampara (String nombreLampara) {
    this.nombreLampara = nombreLampara;
    this.estado = Estado.Apagado;
     numLamps ++;
}

Void encender () {
    this.estado = Estado.Encendido;
}

Estado consulta () {
    return this.estado;
}

Static int numLamps () {
    return numLamps;
}
```

Reto: estado tiene el tipo Estado, y nace con valor Estado.APAGADO

Justificación:

- nombreLampara: tipo String, Se puede mutar y se usa en el constructor. atributo de instancia.
- estado: tipo Estado, es mutable y se inicializa en el constructor como Estado.Apagado. atributo de instancia.
- numLamps: tipo int, atributo de clase, mutable, se incrementa +1 en el constructor.
- ETIQUETA: tipo String, atributo de clase, inmutable, no cambia para ninguna lampara.

- Lampara (String): Constructor de la clase, inicia los atributos y actualiza el contador.
- encender (): método de instancia, es mutable, no recibe Parámetros y no devuelve nada.
- consulta (): método de instancia, solo sirve de consulta para devolver el Estado de la lámpara, regresa un tipo de dato Estado y no recibe Parámetros.
- numLamps (): método para consultar devuelve un tipo static, no recibe Parámetros y es para conocer el número de lamparas creadas.

A4. Código con repetición:

```
Static Int PosicionMinima (int[] temperatura ) {
    int menor = 0;
    For (int i = 1; i < temperaturas.length; i++) {
        if ( temperaturas [i] < temperaturas [menor] {
            menor = i;
        }
    }
    return menor;
}
```

Ejemplo 1: {8, 3, 5} Devuelve 1.

Devuelve 1 Porque comienza menor = 0 que es 8. En i = 1, compara 3 < 8, es verdadero y se actualiza el menor = 1. En i = 2, compara 5 < 3, es falso y por eso se queda con el 1 y lo retorna.

Ejemplo 2: {7} Devuelve 0.

El arreglo es tamaño 1. el for inicia en i = 1 para no comparar (a) mismo elementos, pero la condición 1 < 1 es falsa, el ciclo no se ejecuta ni una vez y retorna el menor = 0.

Ejemplo 3: {-2, -8, -1} Devuelve 1.

Comienza con menor = 0 que es el -2, en i = 1 compara -8 < -2, es verdadero cambia menor = 1. En i = 2 compara -1 < -8, que es falso, retorna el 1 y lo devuelve.

# R5. Códigos con Selección.

```
Static String mantenimiento (int ciclos){
  if (ciclos <= 0){
     return "INVALIDO";
  } else if (ciclos % 24 == 0){
    return "COMPLETO";
  } else if (ciclos % 12 == 0){
    return "REVISION";
  } else if (ciclos % 4 == 0){
    return "BASICO";
  } else {
    return "NINGUNO";
  }
}
```

Renglón 1: Menor o igual que 0.

Devuelve "INVALIDO", Porque es la primera condición que se evalua, si el número es menor o igual a 0 entra aquí, termina la ejecución y devuelve "INVALIDO".

Renglón 2: Divisible entre 24.

Devuelve "COMPLETO" Porque si el número al dividirlo da como residuo 0, quiere decir que es divisible entre 24

Renglón 3: Si no, Divisible entre 12.

Devuelve "REVISION" Por el else if. Si el número no es divisible entre 24 pero si entre 12, se cierra en esta condición y devuelve "REVISION"

Renglón 4: Si no, Divisible entre 4.

Igual que el anterior, si no se cumple lo del 12 ó el 24, se ve si es divisible entre 4, si sí devuelve "BASICO".

Renglón 5: Cualquiera.

Es la estructura final el else, si un número mayor a 0 no cumple ninguno de los anteriores, llega hasta aquí y devuelve "NINGUNO".